Sebenta 04

Containers e Isolamento de Processos

Infraestruturas de Computação na Nuvem | Aula 04

Objeto e utilização deste documento

O presente documento acompanha a apresentação da Aula 04. Cada página reproduz um slide e desenvolve, em texto corrido, a matéria nele condensada, com o detalhe que o tempo de aula não permite e que o estudo posterior exige. A aula é expositiva e não tem componente laboratorial própria, mas retoma as observações feitas no laboratório da Aula 03 e prepara o terreno conceptual da Aula 05, dedicada à orquestração com Kubernetes.

Recomenda-se uma leitura sequencial. A matéria encadeia-se: primeiro compara-se o container com a máquina virtual, depois estudam-se as duas primitivas do núcleo Linux que o tornam possível, em seguida a forma como as imagens são construídas e armazenadas, e por fim a arquitetura da ferramenta Docker, as suas redes, os volumes, as boas práticas e o padrão que garante a portabilidade.

Containers e Isolamento de Processos

Infraestruturas de Computação na Nuvem, Aula 04

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

A Aula 03 terminou com uma observação que ficou por explicar. No laboratório, um container mostrou uma árvore de processos própria, um nome de máquina próprio e um sistema de ficheiros próprio, e no entanto o comando que revela a versão do núcleo devolveu, em Linux, exatamente o mesmo valor que no anfitrião. Não havia máquina virtual nem núcleo duplicado, e ainda assim havia isolamento. Esta aula explica como isso é possível.

A resposta resume-se numa frase que acompanha toda a exposição: um container não é uma máquina virtual pequena, é um processo Linux comum ao qual o núcleo aplicou duas restrições, uma sobre aquilo que o processo vê e outra sobre aquilo que o processo pode consumir. Tudo o que se segue, das imagens em camadas ao Docker e ao padrão OCI, é construído sobre esta ideia simples.

Agenda (1/2)

- Containers vs máquinas virtuais.
- Primitivas Linux de isolamento: namespaces e cgroups.
- Union filesystems, imagens em camadas e as suas limitações.

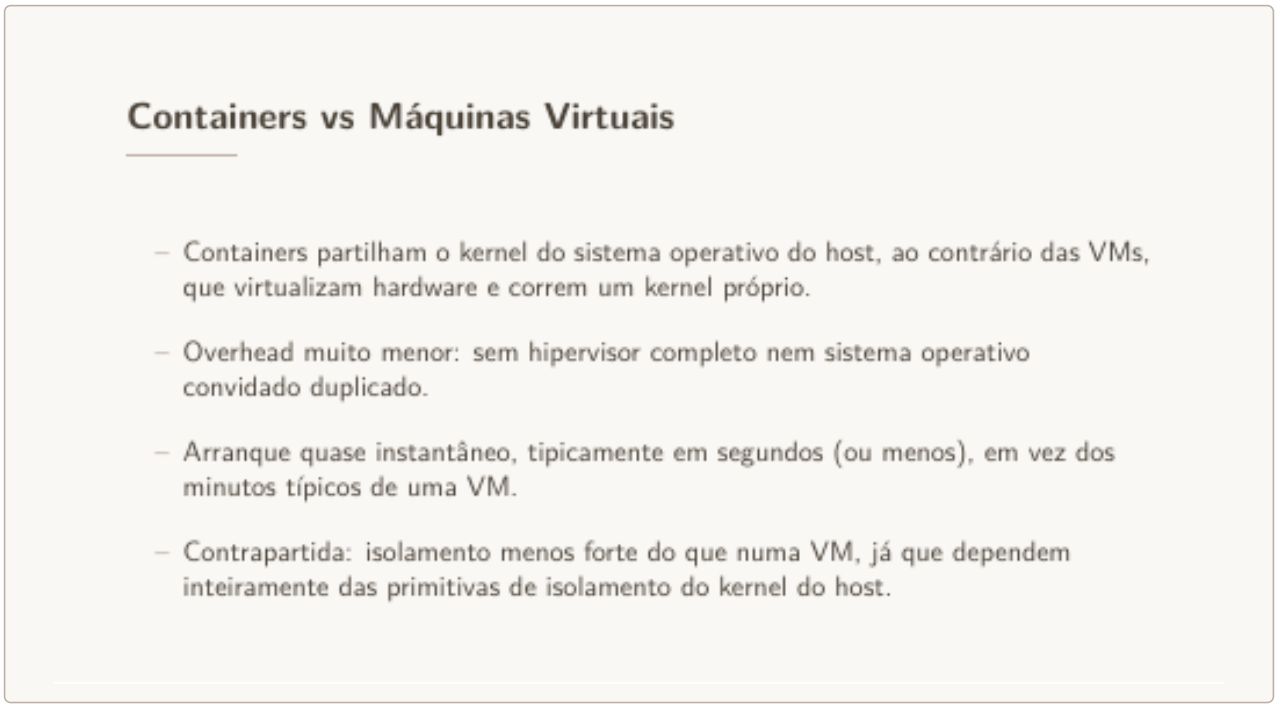
A primeira metade da agenda trata dos alicerces. Começa pela comparação entre containers e máquinas virtuais, que retoma a fronteira de duplicação da Aula 03 e a aplica a este novo modelo. Prossegue com as duas primitivas do núcleo, os namespaces e os cgroups, que são o verdadeiro mecanismo de isolamento. Termina com os sistemas de ficheiros por camadas, que explicam por que razão uma imagem é leve de transportar e rápida de arrancar, e com as limitações que essa solução traz.

Esta parte é a mais exigente do ponto de vista conceptual. Quem a dominar encontrará a segunda metade, mais próxima da prática, muito mais simples de seguir.



A segunda metade da agenda passa do mecanismo para a ferramenta. Apresenta a arquitetura do Docker e o papel de cada um dos seus componentes, a forma como as imagens são distribuídas e versionadas, o modo como os containers comunicam entre si e com o exterior, e a maneira de conservar dados para além da vida de um container.

Seguem-se dois temas de natureza mais crítica. As boas práticas e a segurança lembram que usar containers não dispensa disciplina na sua construção. A distinção entre containers de aplicação e de sistema, e o padrão da Open Container Initiative, fecham a aula com a questão da portabilidade e com a convergência entre containers e máquinas virtuais leves, que reaparece na Aula 05.



Uma máquina virtual reproduz um computador completo. O hypervisor apresenta hardware virtual, e sobre esse hardware corre um sistema operativo inteiro, com o seu núcleo, os seus controladores de dispositivos e os seus serviços de base. Um container reproduz apenas o ambiente de um processo. O núcleo é o do anfitrião, partilhado por todos os containers, e cada container recebe uma vista própria do que esse núcleo gere. Na terminologia da Aula 03, a fronteira de duplicação passa a situar-se acima do núcleo, e não abaixo dele.

As consequências são diretas. Não há sistema operativo convidado a duplicar em memória, pelo que a sobrecarga é muito menor e é possível alojar muitas dezenas de containers onde caberiam poucas máquinas virtuais. Não há sequência de arranque de um sistema operativo, pelo que um container arranca na ordem do segundo, ou menos, porque arrancar um container é criar um processo. Existe também uma restrição que decorre da partilha do núcleo: um container Linux exige um núcleo Linux. Para executar containers Linux em Windows ou macOS recorre-se, como se viu no laboratório, a uma máquina virtual Linux interposta.

A contrapartida é o isolamento mais fraco. Numa máquina virtual, a superfície que separa uma instância de outra é a interface estreita do hypervisor. Num container, é a interface de chamadas de sistema do núcleo, muito mais vasta, e um defeito nessa interface é um defeito partilhado por todos os containers do anfitrião. Daí que a escolha entre os dois modelos seja uma decisão de compromisso, e não de superioridade de um deles.

Namespaces Linux (1/2)

- Namespaces dão a cada processo uma visão isolada de um determinado recurso do sistema, criando a ilusão de máquina dedicada.
- **PID**: árvore de processos própria, isolada da do host.
- **NET**: pilha de rede própria (interfaces, rotas, portas).
- **MNT**: sistema de ficheiros e pontos de montagem próprios.

Um namespace é uma partição lógica de um recurso global do núcleo. Em vez de existir uma única lista de processos, uma única pilha de rede e uma única árvore de ficheiros para todo o sistema, o núcleo mantém várias instâncias de cada uma e coloca cada processo numa delas. O processo vê apenas a instância a que pertence e, como não tem forma de referir o que está fora dela, tem a ilusão de dispor de uma máquina só sua.

O PID namespace dá ao container a sua própria numeração de processos. O primeiro processo lançado dentro dele recebe o número 1, tal como o init de uma máquina, e não vê nenhum dos processos do anfitrião nem dos outros containers. O NET namespace dá-lhe interfaces de rede, tabela de encaminhamento, regras de filtragem e espaço de portas próprios, de modo que dois containers podem escutar ambos na porta 80 sem conflito. O MNT namespace dá-lhe uma tabela de montagens própria, e é por este meio que o container enxerga como raiz do sistema de ficheiros o conteúdo da sua imagem e não o disco do anfitrião.

Os namespaces não são uma invenção do Docker. São uma funcionalidade do núcleo Linux, criada com a chamada `clone` e as suas variantes, e a sua existência pode ser verificada em qualquer processo através da pasta `/proc/<pid>/ns`. O mais antigo é o de montagens, introduzido em 2002, o que explica que o respetivo identificador interno seja simplesmente `NEWNS`.

Namespaces Linux (2/2)

- **UTS**: hostname e domainname próprios.
- **IPC**: mecanismos de comunicação entre processos isolados (semáforos, filas de mensagens).
- **USER**: mapeamento de identificadores de utilizador e grupo, permitindo root dentro do container sem privilégios no host.
- Um container Docker é, na prática, um processo Linux normal ao qual foram atribuídos vários destes namespaces.

O UTS namespace isola o nome da máquina e o nome de domínio. A sigla vem de UNIX Time-sharing System e é herança da estrutura de dados onde estes valores são guardados. É por isso que, dentro de um container, o comando `hostname` devolve um identificador do container e não o nome do portátil, como se observou no laboratório. O IPC namespace isola os mecanismos de comunicação entre processos que vivem na memória do núcleo, como semáforos, filas de mensagens e segmentos de memória partilhada, impedindo que processos de containers distintos interfiram uns com os outros através deles.

O USER namespace é o mais subtil e o mais importante do ponto de vista da segurança. Permite estabelecer uma correspondência entre os identificadores de utilizador e de grupo vistos dentro do namespace e os vistos fora dele. Um processo pode ser root, com identificador 0, no seu interior, e corresponder no anfitrião a um utilizador sem privilégios, com um identificador elevado. Assim, uma fuga do container não entrega ao atacante privilégios de administrador do anfitrião.

Convém registar duas precisões. A primeira é que o Docker, por omissão, não ativa o remapeamento de utilizadores, pelo que o root de um container corresponde ao root do anfitrião, ainda que com capacidades muito reduzidas, como se verá adiante. O remapeamento existe como opção, tal como o modo *rootless*, e ferramentas como o Podman usam-no de raiz. A segunda é que a lista do slide não é exaustiva: o núcleo suporta ainda os namespaces de cgroup e de tempo, este último desde a versão 5.6.

A frase final do slide condensa o conjunto. Um container Docker é um processo Linux normal a que foram atribuídos vários destes namespaces. Não existe no núcleo uma entidade chamada container.

Cgroups: Limitação de Recursos

- Enquanto os namespaces isolam o que um processo *vê*, os cgroups (control groups) limitam o que um processo pode *consumir*.
- Limitação de CPU: quota e período de utilização atribuídos a um grupo de processos.
- Limitação de memória: teto máximo, com possibilidade de terminar o processo (OOM kill) se for excedido.
- Limitação de I/O: controlo de largura de banda de leitura/escrita em disco por grupo.

Os namespaces resolvem o problema da visibilidade, mas deixam por resolver o do consumo. Um processo isolado por namespaces não vê os vizinhos, mas continua a poder esgotar a memória ou o processador da máquina e deixá-los sem recursos. Os control groups, abreviadamente cgroups, respondem a esta segunda questão: organizam processos em grupos e impõem limites e contabilização de recursos a cada grupo.

No processador, o limite exprime-se como uma quota de tempo dentro de um período. Uma quota de 50 000 microssegundos num período de 100 000 equivale a metade de um núcleo de processamento. Quando o grupo esgota a quota, os seus processos ficam suspensos até ao período seguinte, comportamento a que se chama *throttling*. Não há terminação de processos, apenas abrandamento.

Na memória, o comportamento é diferente porque a memória não é um recurso compressível. Se um grupo ultrapassa o teto, o núcleo tenta recuperar páginas e, falhando, invoca o terminador de processos por falta de memória, o OOM killer, que elimina um processo do próprio grupo. O container não é suspenso, é morto. Nos dispositivos de bloco, o controlador de I/O limita a largura de banda ou o número de operações por segundo de leitura e escrita. Existe ainda um controlador do número de processos, que protege o anfitrião contra uma *fork bomb* originada dentro de um container.

Na prática, estes limites são pedidos ao Docker através de opções como `--cpus`, `--memory` e `--pids-limit`, e o Docker traduz-as em valores escritos nos ficheiros do cgroup do container.

Cgroups: Versões e Funcionamento

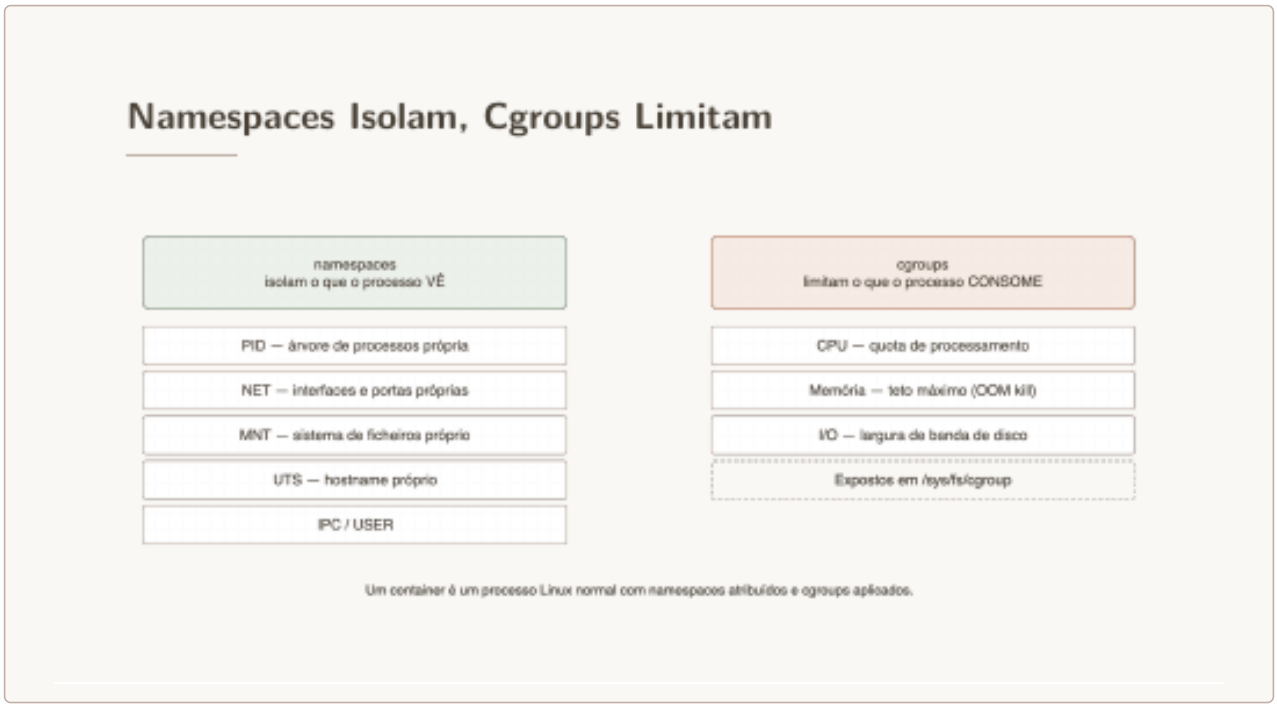
- Os cgroups são expostos pelo kernel através de um sistema de ficheiros virtual, tipicamente em `/sys/fs/cgroup`.
- Escrever valores nesses ficheiros define os limites aplicados a um grupo de processos.
- **cgroups v1**: hierarquias separadas e independentes para cada tipo de recurso (CPU, memória, I/O).
- **cgroups v2**: hierarquia única e unificada, com um modelo de delegação mais coerente; é o modelo usado por omissão nas distribuições Linux recentes.
- Sem cgroups, um único container descontrolado poderia consumir toda a memória ou CPU do host.

Os cgroups são expostos pelo núcleo como um sistema de ficheiros virtual, normalmente montado em `/sys/fs/cgroup`. Cada grupo é uma pasta, cada parâmetro é um ficheiro nessa pasta, e configurar um limite consiste em escrever um valor num ficheiro. Não há ferramenta especial: as ferramentas de linha de comandos e o Docker limitam-se a ler e escrever estes ficheiros. Na demonstração desta aula, o ficheiro `memory.max` dentro do container mostra o teto de memória que lhe foi atribuído, e o valor `max` indica que não há limite.

A tecnologia tem duas gerações. Na primeira, a versão 1, cada tipo de recurso tem a sua própria hierarquia independente, e um mesmo processo pode ocupar posições diferentes em cada uma. Isto tornou a coordenação entre controladores difícil e fonte de inconsistências. A versão 2, declarada estável no núcleo 4.5, em 2016, adotou uma hierarquia única e unificada, em que cada processo pertence a um só grupo e todos os controladores atuam sobre essa árvore. As distribuições recentes, como o Fedora, o Debian 11 e posteriores, o Ubuntu desde a versão 21.10 e o RHEL 9, usam-na por omissão.

A origem dos cgroups é anterior ao Docker. Foram desenvolvidos por engenheiros da Google, que os chamaram inicialmente *process containers*, e integrados no núcleo em 2008. A última frase do slide dá a justificação prática: sem eles, um único container descontrolado esgotaria a memória ou o processador do anfitrião e arrastaria os restantes.

Há uma armadilha clássica. Um cgroup limita o consumo, mas não altera aquilo que o processo observa. Dentro de um container com 256 MiB de limite, ferramentas antigas continuam a ler a memória total do anfitrião em `/proc`, e aplicações que dimensionam as suas estruturas a partir desse valor podem exceder o teto e ser mortas. Por isso as plataformas de execução modernas, como a máquina virtual de Java, passaram a ler os limites do cgroup.



O diagrama organiza a distinção que as duas páginas anteriores desenvolveram. À esquerda, os namespaces respondem à pergunta sobre o que o processo vê: a sua árvore de processos, as suas interfaces e portas, o seu sistema de ficheiros, o seu nome de máquina, os seus canais de comunicação e os seus utilizadores. À direita, os cgroups respondem à pergunta sobre o que o processo consome: processador, memória e largura de banda de disco, expostos em `/sys/fs/cgroup`.

As duas primitivas são independentes e complementares. Um processo pode estar isolado por namespaces sem limite de recursos, e pode estar limitado por cgroups sem qualquer isolamento de visão. Um container resulta de aplicar as duas em simultâneo, e é esse o sentido da frase inferior do diagrama: um container é um processo Linux normal, com namespaces atribuídos e cgroups aplicados.

Vale a pena fixar esta imagem porque ela permite responder, sem decorar, a perguntas que parecem de memória. Quem se pergunte por que razão dois containers conseguem ambos escutar na porta 80 encontra a resposta no NET namespace. Quem se pergunte por que razão um container foi terminado sem aviso encontra a resposta no limite de memória do cgroup. E quem se pergunte por que razão um container arranca em instantes dá com a explicação mais simples: criar namespaces e escrever num cgroup são operações baratas do núcleo, não há sistema operativo a arrancar.

Isolamento Não é Segurança Absoluta

- Todos os containers de um host partilham o mesmo kernel: uma vulnerabilidade no kernel é uma superfície de ataque comum.
- **Capabilities**: o Linux divide os privilégios de root em capacidades individuais (ex. `CAP_NET_ADMIN`); um container deve receber apenas as estritamente necessárias.
- **Seccomp**: filtra as chamadas de sistema (*syscalls*) que um processo pode invocar, reduzindo a superfície exposta do kernel.
- **AppArmor** e **SELinux**: mecanismos de controlo de acesso obrigatório, que restringem o que cada processo pode aceder.
- Quando é exigido isolamento forte (ex. entre clientes diferentes), uma VM continua a ser mais segura do que um container.

A partilha do núcleo é simultaneamente a fonte da eficiência e a fonte do risco. Todos os containers de um anfitrião falam com o mesmo núcleo através da mesma interface de chamadas de sistema, que tem várias centenas de entradas. Um defeito nesse núcleo, ou na forma como um *runtime* configura o isolamento, pode permitir que um processo saia do seu container. Aconteceu, por exemplo, com a vulnerabilidade registada como CVE-2019-5736 no `runc`, que permitia a um container malicioso substituir o executável do próprio *runtime* no anfitrião.

Para reduzir esta superfície, o Linux oferece camadas adicionais de defesa. As *capabilities* decompõem os poderes do utilizador root em dezenas de capacidades individuais, como alterar a configuração de rede ou carregar módulos do núcleo. Um container deve conservar apenas as que lhe são estritamente necessárias, e o Docker retira por omissão a maior parte delas. A opção de executar um container em modo privilegiado devolve-lhe quase todas, e deve ser tratada como equivalente a dar acesso de administrador ao anfitrião.

O *seccomp* filtra as chamadas de sistema que o processo pode invocar. O perfil por omissão do Docker bloqueia uma pequena fracção do conjunto, com 44 chamadas de um total superior a trezentas, escolhidas por serem raramente necessárias e particularmente perigosas. O AppArmor, usado sobretudo em Debian e Ubuntu, e o SELinux, característico de Fedora e RHEL, implementam controlo de acesso obrigatório: uma política externa ao processo determina o que ele pode abrir, executar ou escrever, mesmo que corra como root.

Estas medidas reforçam o isolamento, mas não o transformam no de uma máquina virtual. Para cargas de clientes sem relação de confiança, a prática corrente continua a ser colocar uma máquina virtual entre eles, tal como se concluiu na Aula 03.

Union Filesystems e Imagens em Camadas

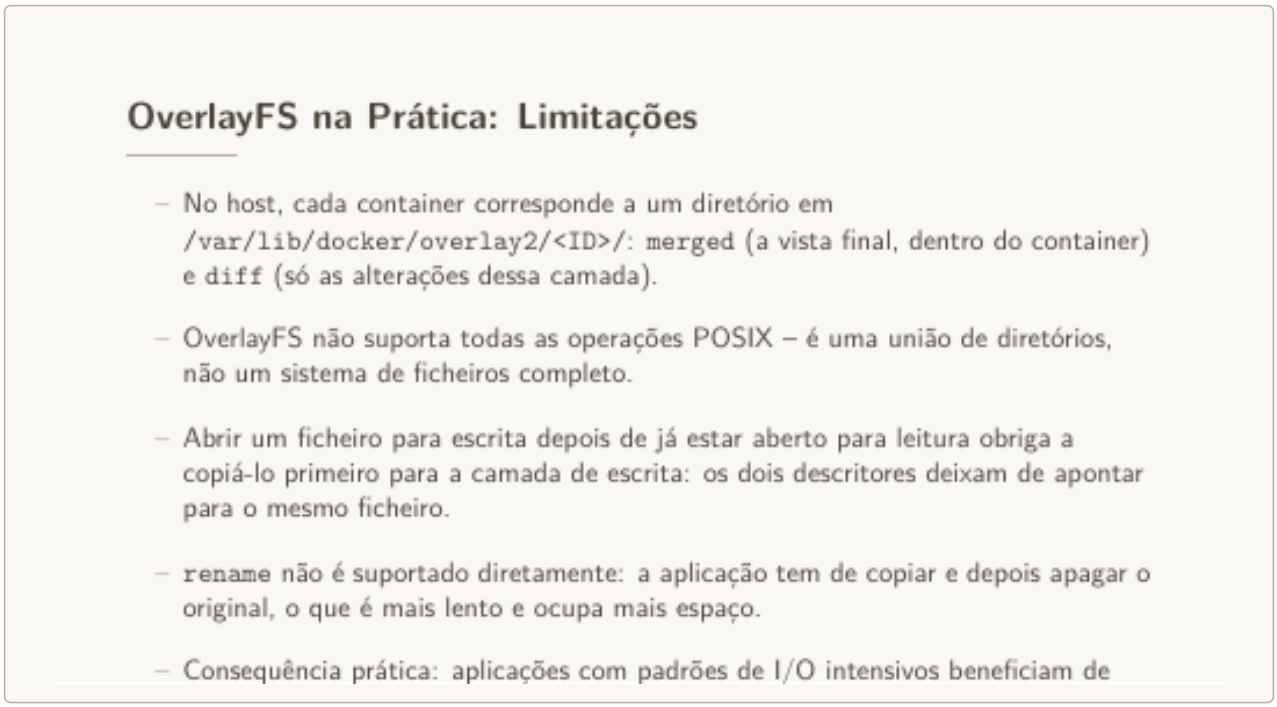
- OverlayFS combina múltiplos diretórios (camadas) numa única vista de sistema de ficheiros.
- Uma imagem Docker é composta por várias camadas só de leitura, empilhadas umas sobre as outras.
- Ao correr um container, é adicionada uma camada de escrita (writable layer) no topo, específica dessa instância.
- Copy-on-write: alterações a ficheiros de camadas inferiores são copiadas para a camada de escrita antes de modificadas, preservando a imagem original.

Uma imagem de container é, no essencial, a árvore de ficheiros que o processo verá como sua raiz, mais alguns metadados. Em vez de a guardar como um bloco único, o Docker guarda-a como uma pilha de camadas só de leitura, cada uma a conter as diferenças relativamente à anterior, e identificada por um resumo criptográfico do seu conteúdo. Duas imagens que partam da mesma base partilham fisicamente as camadas da base, no disco e na rede. Descarregar uma segunda imagem baseada na mesma distribuição transfere apenas as camadas novas.

O OverlayFS é o sistema de ficheiros que combina estas camadas numa única vista. Recebe um conjunto de diretórios inferiores, só de leitura, e um diretório superior, de escrita, e apresenta a sua sobreposição como uma árvore única. Quando um container arranca, o Docker junta às camadas da imagem uma camada de escrita própria, vazia, no topo. Tudo o que o processo escrever ficará nela e nunca nas camadas da imagem, que continuam intactas e disponíveis para outros containers.

O mecanismo que o permite chama-se *copy-on-write*. Ao ler um ficheiro, o processo recebe-o da camada onde ele se encontra. Ao modificá-lo, o ficheiro inteiro é primeiro copiado para a camada de escrita, e só então alterado. A eliminação de um ficheiro que existe numa camada inferior é registada na camada de escrita por uma marca especial, que o torna invisível sem o apagar fisicamente.

A consequência mais visível é que a camada de escrita desaparece com o container. Quando este é removido, tudo o que nela estava perde-se, o que é o ponto de partida para a secção sobre persistência.



No anfitrião, cada container corresponde a uma pasta sob `/var/lib/docker/overlay2`. Nela, a subpasta `diff` contém apenas o que mudou naquela camada, e a subpasta `merged` contém a vista final, a mesma que o processo enxerga como raiz do seu sistema de ficheiros. Isto permite inspecionar, a partir do anfitrião, aquilo que um container escreveu.

O OverlayFS é uma união de diretórios, e não um sistema de ficheiros completo, pelo que não cumpre integralmente a norma POSIX. O primeiro desvio relaciona-se com o *copy-up*. Se uma aplicação abre um ficheiro para leitura e depois o abre para escrita, a segunda abertura obriga a copiá-lo para a camada de escrita, e os dois descritores passam a referir ficheiros diferentes, quando a aplicação esperava que fosse o mesmo. Trata-se de um desvio documentado pelo Docker, raro na prática, mas real.

O segundo desvio respeita à operação de renomear. Renomear um diretório que existe numa camada inferior não é suportado diretamente e devolve um erro de dispositivos distintos. Programas cuidadosos tratam o erro copiando e depois apagando, o que é mais lento e ocupa mais espaço. Outros, menos cuidadosos, falham.

Há ainda uma consequência de desempenho. A primeira escrita num ficheiro grande copia-o integralmente para a camada de escrita, o que é pesado para ficheiros de bases de dados ou de registos. A regra prática do slide decorre daqui: aplicações com muito I/O devem escrever num volume, que é uma pasta do anfitrião montada diretamente no container, fora do OverlayFS.

Arquitetura Docker (1/2)

- **Docker CLI**: interface de linha de comandos usada pelo utilizador.
- **dockerd**: daemon que recebe os pedidos da CLI (ou da API) e gere imagens, redes e volumes.
- **containerd**: runtime de alto nível, responsável pelo ciclo de vida dos containers (criação, execução, paragem).
- **runc**: runtime de baixo nível que efetivamente cria o container, aplicando namespaces e cgroups.

O Docker não é um programa único, mas uma cadeia de componentes com responsabilidades distintas. O utilizador interage com a interface de linha de comandos, o `docker`, que se limita a traduzir os comandos em pedidos à API do daemon, feitos por omissão através de um socket Unix local. O daemon `dockerd` recebe esses pedidos e trata de tudo o que é de alto nível: imagens, redes, volumes e construção.

Para o ciclo de vida dos containers, o `dockerd` delega no `containerd`, um *runtime* de alto nível que gere a transferência e o armazenamento de imagens, a criação, execução e paragem de containers e a ligação aos volumes. O `containerd` não cria o container diretamente. Invoca o `runc`, o *runtime* de baixo nível que efetivamente chama o núcleo, cria os namespaces, configura os cgroups, monta o sistema de ficheiros e lança o processo. Depois de cumprida esta tarefa, o `runc` termina. O processo do container passa a ter como pai um pequeno processo auxiliar, o `containerd-shim`, que mantém as ligações de entrada e saída e comunica o código de saída.

A separação foi feita a partir da versão 1.11, em 2016, e tem uma consequência prática relevante: o daemon pode ser reiniciado sem terminar os containers em execução, porque estes não são seus filhos diretos. O `containerd` foi mais tarde doado à CNCF e o `runc` à Open Container Initiative, que se estuda mais adiante. É por isso que o Kubernetes pode usar o `containerd` diretamente, sem o Docker, como se verá na Aula 05.

Arquitetura Docker (2/2)

- Um Dockerfile descreve, em passos declarativos, como construir uma imagem (imagem base, dependências, código, comando de arranque).
- Cada instrução do Dockerfile gera, em regra, uma nova camada da imagem.
- O comando `docker build` executa estes passos e produz a imagem final.
- A imagem resultante pode depois ser instanciada em um ou mais containers com `docker run`.

Um Dockerfile é a descrição declarativa de como construir uma imagem. Indica a imagem de partida, as operações a executar sobre ela, os ficheiros a incluir e o comando que o container deve executar ao arrancar. Por ser um ficheiro de texto, pode ser guardado num repositório de código, revisto e reproduzido, o que faz da imagem um artefacto auditável e não o resultado de uma sequência de ações manuais.

Cada instrução que altera o sistema de ficheiros, nomeadamente `RUN`, `COPY` e `ADD`, produz uma nova camada. As restantes, como `ENV`, `EXPOSE` ou `CMD`, registam apenas metadados na imagem. É por isso que o slide diz que cada instrução gera *em regra* uma camada. O `docker build` percorre as instruções por ordem, reutiliza as camadas que não mudaram e constrói as restantes.

A distinção entre imagem e container é análoga à que a Aula 03 fez entre o ficheiro de disco virtual e a máquina virtual em execução. A imagem é um modelo imutável, e o container é uma instância em execução desse modelo, com a sua camada de escrita. De uma mesma imagem podem sair tantos containers quantos se pretenda, com o comando `docker run`.

Uma distinção útil no Dockerfile é a que existe entre `ENTRYPOINT` e `CMD`. O primeiro fixa o executável do container, o segundo fornece os argumentos por omissão, que podem ser substituídos na linha de comandos.



Um registry é o serviço que armazena e distribui imagens. A operação de `pull` descarrega uma imagem, a de `push` envia-a. O Docker Hub é o registry público de referência e aloja as imagens oficiais das distribuições e dos principais programas, mas existem outros, como o GitHub Container Registry e o Quay, e as organizações mantêm frequentemente registries privados para imagens internas, por razões de confidencialidade e de controlo.

Uma referência a uma imagem tem a forma `registry/repositório:tag`. A *tag* identifica uma versão, como em `nginx:1.25`. Há duas ressalvas essenciais. A primeira é que uma tag é uma etiqueta móvel: quem publica a imagem pode reapontá-la para conteúdo diferente, e a mesma tag pode, por isso, referir imagens distintas em momentos distintos. A segunda é que `latest` não significa a versão mais recente. É apenas a tag que o Docker assume quando nenhuma é indicada, e o seu conteúdo é o que o autor quiser.

Para reprodutibilidade exata existe o *digest*, o resumo criptográfico do conteúdo, que é imutável: referir uma imagem por digest garante que se obtém precisamente os mesmos bytes. Em produção, fixar versões explícitas ou digests evita que uma reconstrução ou uma reinstalação traga, sem aviso, uma versão diferente da aplicação.

Redes em Docker

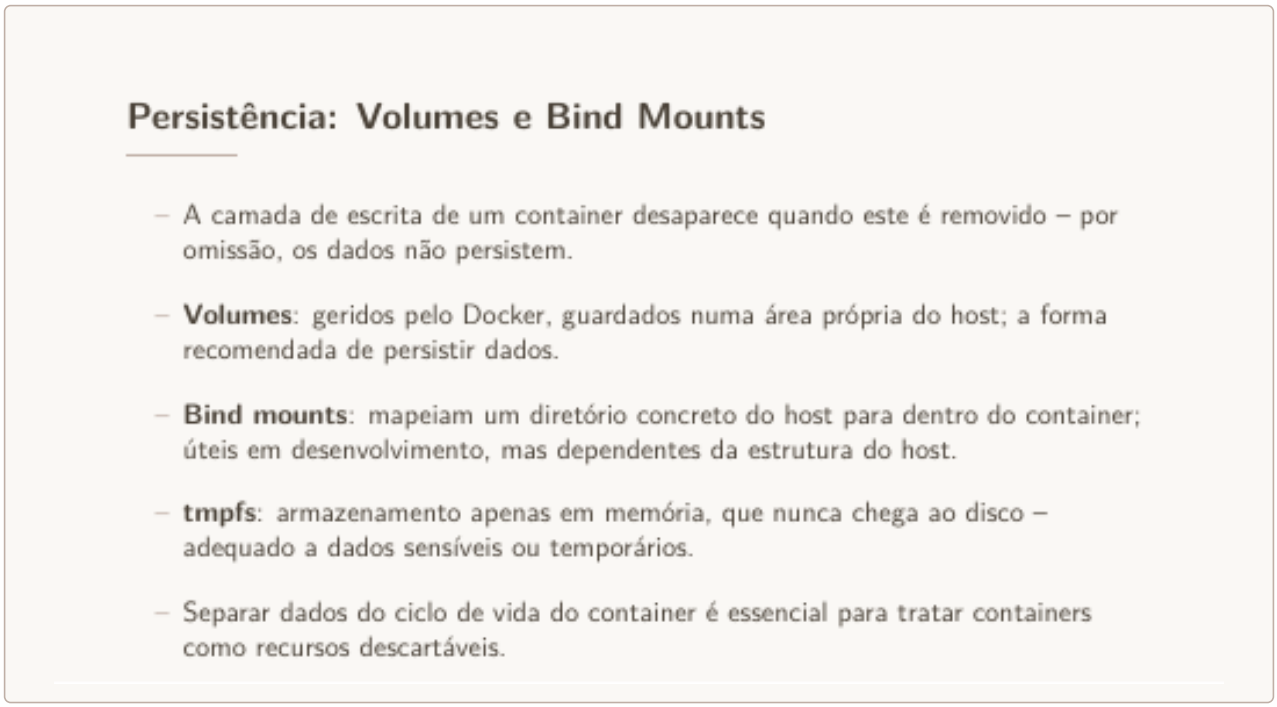
- **bridge**: rede virtual isolada no host, usada por omissão; os containers comunicam entre si e saem para o exterior por NAT.
- Numa rede bridge *personalizada*, o Docker fornece resolução de nomes: um container alcança outro pelo seu nome.
- **host**: o container usa diretamente a pilha de rede do host, sem isolamento de rede (sem namespace NET próprio).
- **none**: sem qualquer interface de rede, para cargas de trabalho totalmente isoladas.
- Publicação de portas (`-p 8080:80`) mapeia uma porta do host para uma porta do container.

O modelo de rede do Docker assenta nos mesmos namespaces de rede já estudados. Cada container tem o seu NET namespace e é ligado à rede do anfitrião por um par de interfaces virtuais, em que uma ponta fica no container e a outra num comutador virtual do anfitrião, a bridge.

O modo *bridge* é o de omissão. Os containers recebem endereços numa rede privada, tipicamente 172.17.0.0/16 na rede por omissão, comunicam entre si por esse endereço e saem para o exterior por tradução de endereços. Uma limitação importante da rede bridge por omissão é a ausência de resolução de nomes. Numa rede bridge criada pelo utilizador, o Docker fornece um servidor DNS interno e um container alcança outro simplesmente pelo nome, razão pela qual é a opção recomendada para aplicações com vários componentes.

No modo *host*, o container deixa de ter NET namespace próprio e usa diretamente a pilha de rede do anfitrião. Ganha-se desempenho e perde-se isolamento: não há mapeamento de portas, e dois containers não podem escutar na mesma. No modo *none*, o container não tem qualquer interface além da de *loopback*, adequado a cargas que não devem comunicar.

Para tornar um serviço acessível do exterior, publica-se uma porta com a opção `-p 8080:80`, em que o primeiro número é a porta do anfitrião e o segundo a do container. Convém saber que, por omissão, a publicação é feita em todas as interfaces do anfitrião e que as regras que o Docker insere na filtragem podem contornar a firewall do sistema. Quando o serviço só deve ser acessível localmente, indica-se o endereço de *loopback* na publicação.



A camada de escrita de um container é efémera por construção. Desaparece quando o container é removido, e é por isso que, por omissão, os dados escritos num container não persistem. Esta característica não é um defeito, é a base do modelo: um container é descartável e substituível, e os dados que importam têm de viver noutro sítio.

Os volumes são a forma recomendada. São criados e geridos pelo Docker numa área própria do anfitrião, sob `/var/lib/docker/volumes`, têm um nome e um ciclo de vida independente do dos containers, e podem ser montados em vários deles. Quando um volume vazio é montado numa pasta que já tem conteúdo na imagem, esse conteúdo é copiado para o volume na primeira utilização. É o que acontece, na demonstração desta aula, com os dados do Redis guardados na pasta `/data`.

Os *bind mounts* montam uma pasta concreta do anfitrião dentro do container. São cómodos em desenvolvimento, porque permitem editar o código no anfitrião e vê-lo de imediato no container, mas criam dependência da estrutura de pastas e das permissões do anfitrião, o que prejudica a portabilidade. O `tmpfs` é um sistema de ficheiros mantido apenas em memória: nada chega ao disco, o que o torna adequado a dados sensíveis e temporários, como chaves ou ficheiros de sessão, que desaparecem com o container.

Separar os dados do ciclo de vida do container permite tratá-lo como um recurso descartável: pode ser destruído e recriado a partir da imagem, sem perda de informação, quando é preciso atualizar a aplicação ou recuperar de uma falha.

Boas Práticas na Construção de Imagens

- Usar imagens base minimalistas (ex. Alpine, ou variantes *slim*) para reduzir tamanho e superfície de ataque.
- **Multi-stage builds**: compilar numa fase intermédia e copiar apenas o resultado final para a imagem entregue.
- Ordenar as instruções do Dockerfile do menos ao mais volátil, para aproveitar a cache de camadas (ex. instalar dependências antes de copiar o código).
- `.dockerignore`: evita enviar ficheiros desnecessários para o contexto de build.
- Não incluir segredos (passwords, chaves) na imagem: ficam registados nas camadas e são recuperáveis.
- Correr a aplicação com um utilizador sem privilégios, em vez de root.

A construção de uma imagem é uma atividade de engenharia com consequências sobre o tamanho, a velocidade e a segurança. A primeira regra é partir de uma base minimalista. As variantes baseadas em Alpine ou as *slim* reduzem o tamanho e, com ele, a superfície de ataque, porque um programa que não está instalado não pode ser explorado. Há que ter em conta que o Alpine usa uma biblioteca C diferente da das distribuições mais comuns, o que pode causar incompatibilidades com programas pré-compilados.

As *multi-stage builds* separam a construção da execução. Compila-se numa primeira fase, que contém compiladores e dependências de desenvolvimento, e copia-se para a imagem final apenas o resultado. A imagem entregue deixa de transportar ferramentas que só serviram para construir.

A ordem das instruções importa por causa da cache de camadas. Quando uma instrução muda, todas as seguintes têm de ser reconstruídas. Por isso colocam-se primeiro as instruções que mudam raramente, como a instalação de dependências, e só depois a cópia do código, que muda a cada alteração. O ficheiro `.dockerignore` impede que ficheiros desnecessários, como repositórios de controlo de versões ou ambientes virtuais, sejam enviados para o contexto de construção.

Quanto à segurança, há duas regras que não admitem exceção. Os segredos não devem ser incluídos na imagem: apagar um ficheiro numa camada posterior não o remove das camadas anteriores, e qualquer pessoa com acesso à imagem pode recuperá-lo. E a aplicação deve correr com um utilizador sem privilégios, declarado com a instrução `USER`, para que a exploração de uma falha da aplicação não conceda, de imediato, privilégios de administrador dentro do container.

Containers de Aplicação vs de Sistema

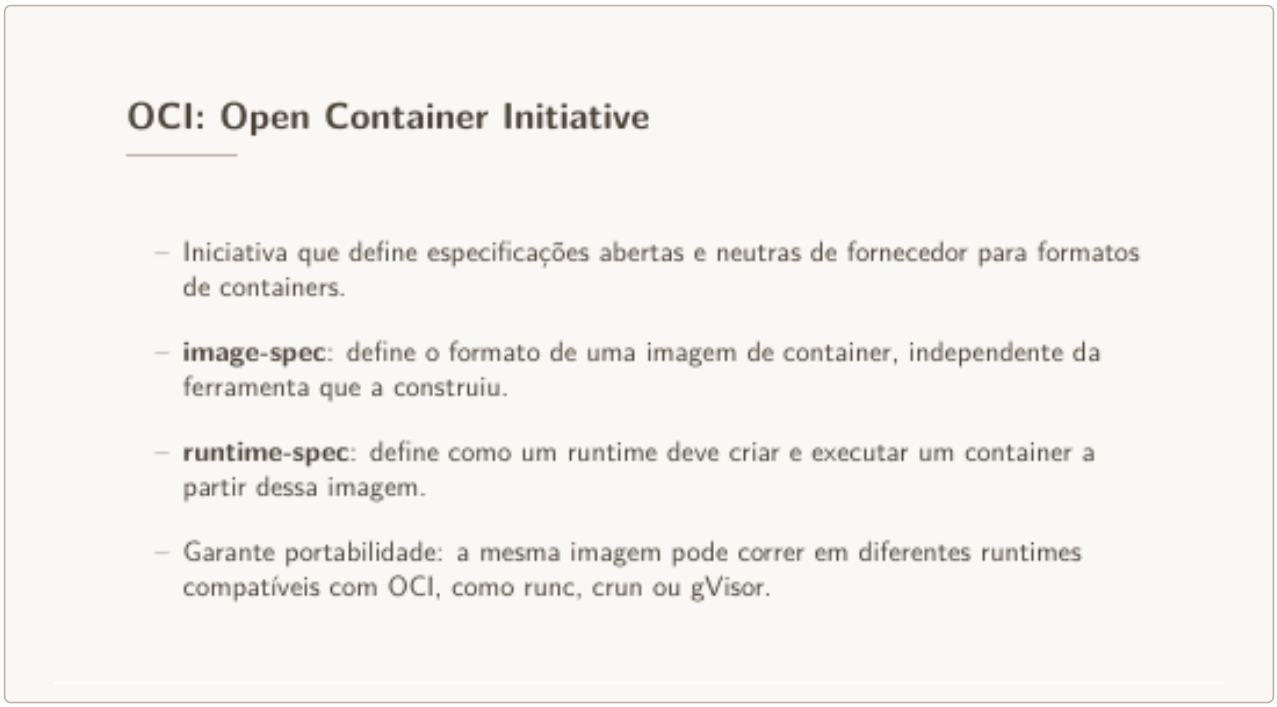
- **Container de aplicação** (Docker): corre tipicamente um único processo/serviço; é efémero e descartável.
- Não tem um sistema de init completo; o ciclo de vida do container é o ciclo de vida desse processo.
- **Container de sistema** (LXC, visto na Aula 03): comporta-se como uma máquina Linux completa, com `systemd` e múltiplos serviços.
- Ambos usam as mesmas primitivas do kernel (namespaces e cgroups) – a diferença está na forma de utilização, não na tecnologia de base.
- Nos laboratórios desta UC usamos os dois: LXC no Proxmox para máquinas de sistema, Docker para empacotar aplicações.

Embora usem as mesmas primitivas do núcleo, os containers podem ser utilizados de duas maneiras muito diferentes. Um container de aplicação, o modelo do Docker, executa um único processo ou serviço, e o ciclo de vida do container coincide com o desse processo: quando o processo termina, o container termina. Não tem sistema de init completo, é efémero e foi concebido para ser destruído e recriado sem cerimónia.

Esta característica tem uma implicação técnica menos óbvia. O processo da aplicação ocupa o lugar do PID 1 no seu namespace e herda as responsabilidades especiais desse papel: tratar os sinais de terminação e recolher processos órfãos. Aplicações que não foram escritas a pensar nisso podem ignorar o sinal de paragem ou acumular processos zombie, e a opção `--init` do Docker existe para inserir um pequeno processo que cumpre essas funções.

Um container de sistema, como os do LXC referidos na Aula 03, comporta-se como uma máquina Linux completa. Arranca com `systemd`, executa vários serviços, tem utilizadores, tarefas agendadas e é administrado como se fosse uma máquina virtual, mas sem núcleo próprio. É o modelo adequado quando se pretende migrar uma máquina existente com o mínimo de alterações.

A diferença não está na tecnologia, está na filosofia de utilização. Nos laboratórios da unidade curricular usam-se os dois: o LXC no Proxmox para máquinas de sistema, e o Docker para empacotar aplicações.



Quando o Docker se tornou popular, surgiu o risco de o formato dos containers ficar preso a um único fornecedor. Em 2015, a Docker e outras empresas fundaram, sob a Linux Foundation, a Open Container Initiative, com o objetivo de definir especificações abertas e neutras. A Docker doou à iniciativa o formato de imagem e o `runc`, que passou a ser a implementação de referência.

A *image-spec* define o formato de uma imagem: um manifesto, uma configuração e um conjunto de camadas, cada uma delas um arquivo do sistema de ficheiros. O formato é independente da ferramenta que construiu a imagem. A *runtime-spec* define como um *runtime* deve criar e executar um container a partir de uma pasta com o sistema de ficheiros e um ficheiro de configuração que descreve os namespaces, os cgroups, as capabilities e o processo a executar. Existe ainda uma terceira especificação, a de distribuição, que normaliza a API dos registries.

O resultado é portabilidade. Uma imagem construída com o Docker pode ser executada por outro *runtime* compatível: o `runc`, o `crun`, escrito em C e mais leve, ou o gVisor, que se estuda a seguir. Do mesmo modo, uma imagem pode ser descarregada de qualquer registry e executada por ferramentas diferentes do Docker, como o Podman, que dispensa um daemon, ou o CRI-O, usado em clusters Kubernetes.

Para o utilizador, isto significa que a imagem passa a ser a unidade de distribuição, independente do motor que a vai executar. Para o engenheiro de infraestrutura, significa que a escolha do *runtime* se torna uma decisão que pode ser revista sem alterar as aplicações.

Containers como Máquinas Virtuais Leves

- A fronteira entre container e VM (Aula 03) está a esbater-se: se o runtime e o formato de imagem seguem OCI, a implementação por baixo deixa de ser relevante para quem usa o container.
- **Kata Containers**: implementa a mesma interface OCI, mas cada "container"corre dentro de uma micro-VM leve (com o seu próprio kernel), combinando a portabilidade de Docker com o isolamento de hardware de uma VM.
- **gVisor**: outra abordagem híbrida – intercepta as *syscalls* do container num kernel de utilizador, sem chegar a expor o kernel real do host.
- Motivação: cargas multi-tenant (ex. funções de clientes diferentes no mesmo host) em que o isolamento fraco de um container tradicional é insuficiente.

Se o formato da imagem e a interface do *runtime* são normalizados, a implementação que está por baixo deixa de ser relevante para quem usa o container. Esta observação abriu caminho a soluções que mantêm a interface de um container e substituem o mecanismo de isolamento por outro mais forte, esbatendo a fronteira entre containers e máquinas virtuais estudada na Aula 03.

O Kata Containers cumpre a interface OCI, mas cada container, ou cada grupo de containers, corre dentro de uma máquina virtual muito leve, com o seu próprio núcleo. Combina a portabilidade do modelo Docker com o isolamento por hardware de uma máquina virtual. Resultou da fusão de dois projetos anteriores, o Clear Containers da Intel e o runV da Hyper. O custo é um arranque ligeiramente mais lento e um consumo de memória maior do que o de um container convencional.

O gVisor, desenvolvido pela Google, segue outro caminho. Interpõe entre o container e o núcleo real um núcleo escrito em espaço de utilizador, que intercepta as chamadas de sistema e as trata ele próprio, expondo ao anfitrião uma superfície muito menor. O custo é algum desempenho em cargas com muitas chamadas de sistema e a compatibilidade parcial com as chamadas menos usadas. Uma terceira peça deste panorama é o Firecracker, da Amazon, uma micro máquina virtual sobre KVM concebida para arrancar em fração de segundo, e usada em serviços como o AWS Lambda.

A motivação é sempre a mesma: cargas multi-cliente, como as funções de clientes distintos executadas no mesmo anfitrião, em que o isolamento de um container tradicional é insuficiente e o custo de uma máquina virtual completa é excessivo.

Síntese da Aula (1/2)

- Containers partilham o kernel do host, oferecendo menor overhead e arranque quase instantâneo face a VMs.
- Namespaces isolam o que um processo vê (PID, NET, MNT, UTS, IPC, USER); cgroups limitam o que consome (CPU, memória, I/O).
- OverlayFS e imagens em camadas permitem reutilização eficiente de camadas comuns entre imagens, à custa de limitações POSIX (escrita concorrente, rename) que justificam o uso de volumes para I/O intensivo.

A primeira metade da síntese reúne os alicerces. Os containers partilham o núcleo do anfitrião e por isso têm menor sobrecarga e arranque quase instantâneo face às máquinas virtuais, ao preço de um isolamento mais fraco.

O isolamento resulta de duas primitivas independentes. Os namespaces determinam o que o processo vê, nas dimensões de processos, rede, montagens, nome de máquina, comunicação entre processos e utilizadores. Os cgroups determinam o que o processo consome, em processador, memória e disco. Um container é a aplicação conjunta das duas a um processo comum.

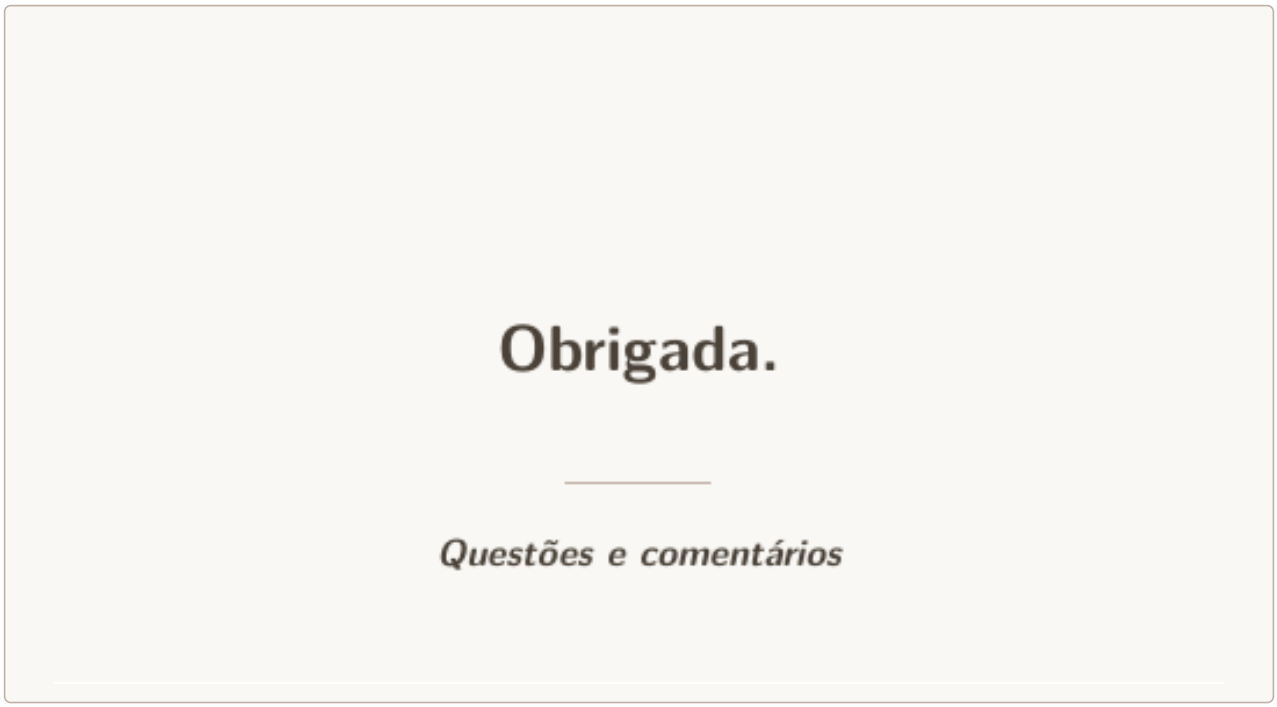
As imagens em camadas, servidas pelo OverlayFS, permitem reutilizar as camadas comuns entre imagens e arrancar containers sem copiar dados. O preço são limitações no cumprimento da norma POSIX, nomeadamente na abertura concorrente e na renomeação, que justificam o uso de volumes para cargas de I/O intensivo.



A segunda metade da síntese reúne o que respeita à ferramenta e à sua utilização. O Docker articula o `dockerd`, o `containerd` e o `runc` para transformar um Dockerfile numa imagem e essa imagem num container em execução. Os registries distribuem imagens versionadas por tags, e o digest oferece a referência imutável quando a reprodutibilidade o exige. As redes bridge tratam da comunicação e os volumes tratam da persistência.

O isolamento dos containers não é absoluto. As capabilities, o seccomp e o AppArmor ou SELinux reforçam-no, mas uma máquina virtual continua a isolar melhor, e é por essa razão que soluções como o Kata Containers e o gVisor aproximam as duas abordagens. A norma OCI garante a portabilidade das imagens entre *runtimes* diferentes.

Quem tenha acompanhado a aula deve conseguir explicar, sem consultar notas, por que razão o comando `ps` dentro de um container mostra o processo principal com o número 1 e o comando `docker top`, executado no anfitrião, mostra o mesmo processo com outro número.



Conclui-se a exposição da Aula 04. A aula seguinte coloca uma questão que a de hoje deixa em aberto: o Docker gere containers num único anfitrião, mas não decide onde correr cada um, nem reage quando o anfitrião falha. A resposta é a orquestração, e o Kubernetes, que a Aula 05 apresenta, retoma muitos dos conceitos aqui estudados, a começar pelo Pod, que é na prática um conjunto de containers que partilham um NET namespace.